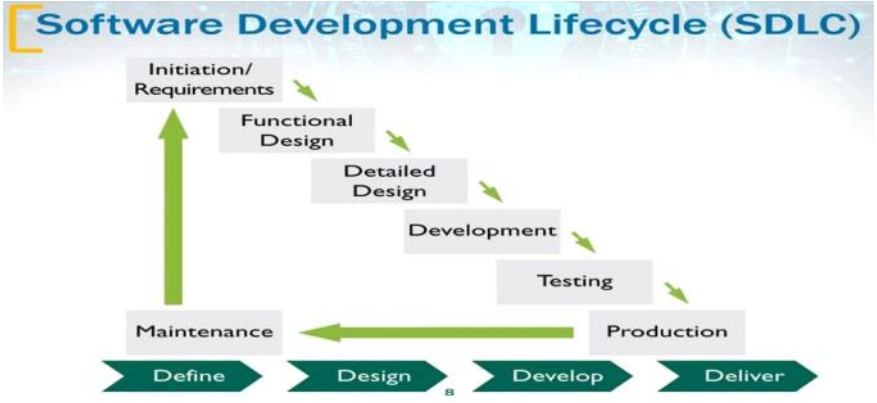
Saturday, February 27, 2021 8:54 PM

Domain 8: Software Development Security

- **Understand and integrate security in the Software Development Life Cycle (SDLC)**
  - The Systems Development Life Cycle (SDLC, also called the Software Development Life Cycle or simply the System Life Cycle) is a system development model
  - The traditional software development model is known as the SDLC (system development life cycle). SDLC is a linear (sequential) model
  - SDLC- System / Software Development Life Cycle (SDLC) - Methodology how to develop applications - 7 Phases
  - SLC - System Life cycle - methodology how to manage applications - 9 Phases (7 Phases of SDLC + 2 phases of SLC), SDLC is part of SLC
  - Benefits of integrating security into the System Development Life Cycle
    - Early identification and mitigation of security vulnerabilities
    - Awareness of potential engineering challenges caused by mandatory security controls
    - Identification of shared security services and reuse of security strategies and tools that will reduce development costs
    - Facilitation of informed executive decision making through the application of a comprehensive risk management process in a timely manner
    - Documentation of important security decisions made during the development.
  - **If you introduce security initially, it's less expensive than introducing security at a later stage. - Remember from the exam point of view.**
  - SDLC Phases - Security must be parallel which needs to be followed in every phase, It's good to introduce the security in functional requirements.
  - **Software Development Lifecycle (SDLC) Phases**
    - Typically, these are considered to be the most basic phases of the SDLC:
      - Project initiation and planning
      - Functional requirements definition
      - System design specification
      - Development and implementation
      - Documentation and common program controls
      - Testing and evaluation control (certification and accreditation)
      - Transition to production(implementation)
  - The System Lifecycle (SLC) covers the life of the system, beyond putting the system into production. Placing the system into production is where the SDLC ends,
    - SLC continues to include two additional phases:
      - Operations and maintenance support (post-installation)
      - Decommissioning/disposal and system replacement

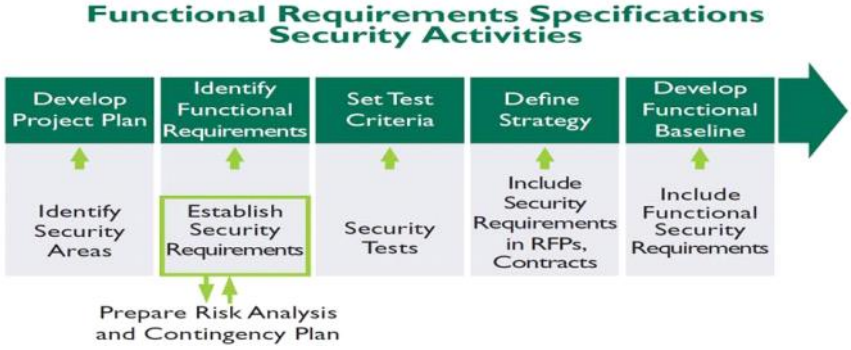


- **Phase 1 - Project Initiation & Planning**
  - **Contingency Planning most important in the Initiation phase**
  - Projects usually start with an idea, a vision, or some conceptual objective.
  - Justification for the undertaking of the project needs to be formulated.
  - Management approval for the project is based on this project plan document as all undertakings should be cost-justified. During this phase is where security must also be involved, as understanding the security requirements needs to begin here.
  - **Security activities need to be done in parallel with project initiation activities and with every single phase guided by the methodology used**.
  - **Reviewing Potential Security Checklist in the Project Initiation Phase**



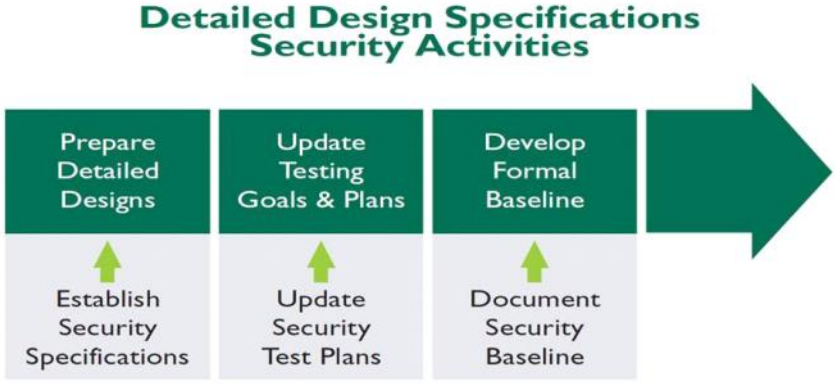
- Establish user requirements --- Determine Security Requirements
- Identify all user controls --- Define Security Strategy
- Select Approve / Approach --- Conduct Risk Analysis
- Management approval --- Security activity should be done in parallel

- **Phase 2 - Functional requirements definition**
  - **Which phase we establish security requirement - Functional Requirement phase**
  - The project management and systems development teams will conduct a comprehensive analysis of current and possible future functional requirements to ensure that the system will meet the owner and organization's needs.
  - The teams also review the documents from the project initiation phase and make any revisions or updates as needed.
  - **This is the phase where security requirements are identified**
  - Security requirements should be formalized as well by involving all parties, especially the owners, compliance, privacy, business-facing functions, and other stakeholders that are informed of the security requirements that the system needs to be able to address.



  - Comprehensive analysis
  - Ensure the system will meet end-user needs
  - Stakeholder expectations & developer understand the requirements
  - Security requirement needs to be considered in this phase

- **Phase 3 - System Design Specifications**
  - **Function is translated to design, Threat Modeling is performed in Design Phase**
  - This phase includes all activities related to designing the system and software. In this phase, the system architecture, system outputs, and system interfaces are designed. Data input, data flow, and output requirements are established and security features are designed into design specification, based on the requirement that we have understood in the previous phase.
  - Prepare the threat modeling (STRIDE & PASTA)



- **Phase 4 - Development and Implementation**
  - **Secure Coding comes in the development stage**
  - During this phase, the source code is generated, test scenarios and test cases are developed, unit and integration testing are conducted, and the program and system are documented for maintenance and turnover to acceptance testing and production.
  - As well as general care for software quality, reliability, and consistency of operation, particular care should be taken to ensure that the code is analyzed to eliminate common vulnerabilities that might lead to security exploits and other risks.
  - Adequate time needs to be allocated for the proper code review before getting into further phases of the SDLC.
  - **In this phase, particular care for software quality, reliability, and consistency of operation needs to be addressed.**
  - **Unit Testing, integration testing, system testing on the functional level are performed in this stage.**



- **Phase 5 - Documentation and Common Program Control**
  - These are controls used when editing the data within the program, the types of logging the program should be doing, and how the program versions should be stored.
  - A large number of such controls may be needed, including tests and integrity checks for:
    - Program/application
    - Operating instructions/procedures
    - Utilities
    - Privileged functions
    - Job and system documentation
    - Components – hardware, software, files, databases, reports, users Restart and recovery procedures
    - Common program controls
    - Edits such as syntax, reasonableness (sanity), range checks, and check digits
    - Logs (who, what, when), Timestamps, Before and after images
- **Phase 6 - Testing and Evaluation Controls**
  - Testing with known good data is also required to ensure the application and system react as expected. A very important rule to always follow is to ensure that live production data should never be used as it may impact privacy and need-to-know policies.
    - Test data should include the following
      - Data at the ends of the acceptable data ranges
      - Various points in between
      - Data beyond expected / Allowable data points
    - Test With
      - Known good data
      - Never live production data
      - Sanitize test data to ensure that sensitive production data is not exposed to test members that should not be allowed to view sensitive data
      - Static Mask of the data, we need to use production data.
  - Clear and adequate segregation between testing and production environments must be maintained at all times.
  - Management and stakeholders should be informed of the test results, and they also need to formally acknowledge the results of the test
  - Certification initiate in Testing Evaluation phase
- **Acceptance**
  - **It Confirms that user needs have been met by the supplied solution**
  - The acceptance phase is one of the most important as this is where we ensure the system does what it is supposed to. But this also includes security capabilities.
  - The very important goal of security testing is to ensure the application meets its security requirements and specifications that were outlined in previous phases.
  - To ensure proper and valuable testing, the application should be tested in an environment that simulates as much as possible, the actual production environment.
  - The security testing strives to uncover all design and implementation flaws that would allow a user to violate the software security policy and requirements. To ensure test validity, one should test the application in an environment that stimulates the production environment. This should include a security certification package and any user documentation. This is the first phase of what is commonly referred to as the certification and accreditation (C&A) process, or the security authorization process.
  - Certification and Accreditation
    - Certification and accreditation are related, but they are not simply two steps in a single process
      - Technical evaluation of a product
      - Management acceptance is part of Accreditation
      - Accreditation:- Transition to production
      - Certification :- Begin in Development & implantation
    - Certification and Accreditation are sometimes referred to as “Security Authorization.”
    - Certification is defined as the formal process of evaluating the security capabilities of the software or system against a predetermined set of security standards or policies.
    - Certification can also examine how well the system performs its intended functional requirements related to security.
    - The certification or evaluation document should contain an analysis of the technical and non-technical security features and countermeasures and the extent to which the software or system meets the security requirements for its mission and operational environment.
    - A certifying officer then verifies that the software has been tested and meets all applicable policies, regulations, and standards for securing information systems. Any exceptions are noted for the accreditation or authorizing official
    - Security activities verify that the data conversion and data entry are controlled, and only those who need to have access are allowed on the system.
- **Phase 7 - Transition to Production/Implementation**
  - In this phase, the new system is transitioned from the acceptance phase into the live production environment
  - Management, after reviewing the certification, authorizes the software or system to be implemented in production status, in a specific environment, for a specific period. There are two types of accreditation: provisional and full.

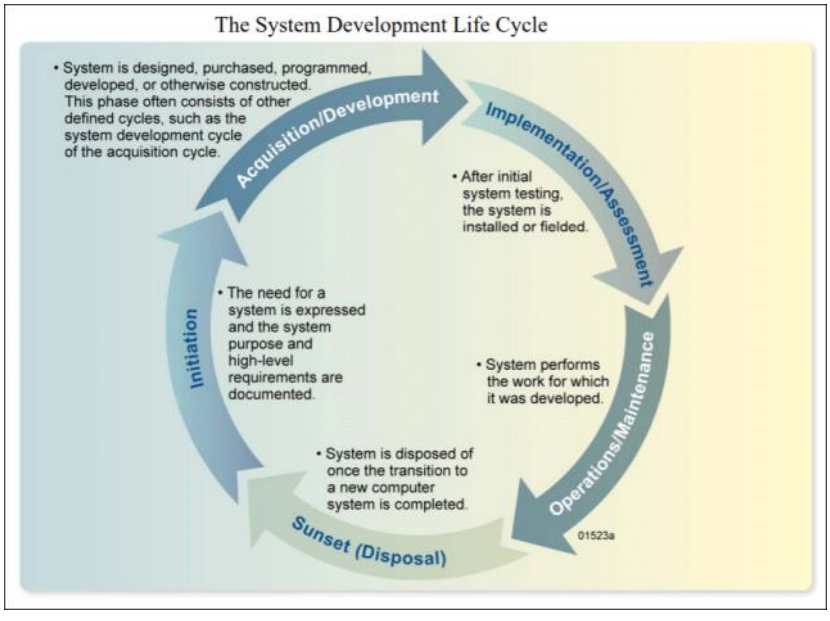
- **Provisional accreditation** is for a specific period and outlines required changes to the applications, system, or accreditation documentation.
- **Full accreditation** implies that no changes are required for making the accreditation decision. Note that management may choose to accredit a system that has failed certification or may refuse to accredit a system even if it has been certified correct

- Activities during this phase may include **obtaining security accreditation, providing training, awareness**, and education to the new users according to the implementation and training requirements, implementing the system, including installation and data conversions; and, if necessary, conducting any parallel operations



- **Phase 8 - Operations and maintenance support (post-installation)**
  - The activities that need to be done here are monitoring the performance of the system on a regular basis but also ensuring the continuity of operations.
  - During operations and maintenance, the organization also needs to manage and prevent system problems, recovering from system problems, and implementing system changes.
  - This includes detecting defects or weaknesses, managing and preventing system problems, recovering from system problems, and implementing system changes.
  - The operating security activities during this phase include testing backup and recovery procedures, ensuring proper controls for data and report handling, and ensuring the effectiveness of security processes.
  - During the maintenance phase, periodic risk analysis and recertification of sensitive applications are required when significant changes occur. Significant changes include a change in data sensitivity or criticality, relocation or major change to the physical environment, new equipment, new external interfaces, new operating system software, and new application software.
  - Throughout the operation and maintenance phase, it is important to verify that any changes to procedures or functionality do not disable or circumvent the security features. Also, someone should be assigned the task of verifying compliance with applicable service-level agreements according to the initial operational and security baselines
- **Phase 9 - Revisions and System Replacement**
  - While systems may be in production mode, the hardware and software baselines should be subject to periodic evaluations and audits.
  - As the system is now in production, any changes that need to be made to the application must follow the same SDLC and be done in a change management system.
  - Revision reviews and approvals should include security planning and procedures. Periodic application audits should be conducted and include documenting security incidents when problems occur.
  - Documenting system failures is a valuable resource for justifying future system enhancements.

- **NIST SDLC Model (EXAM IMP)**

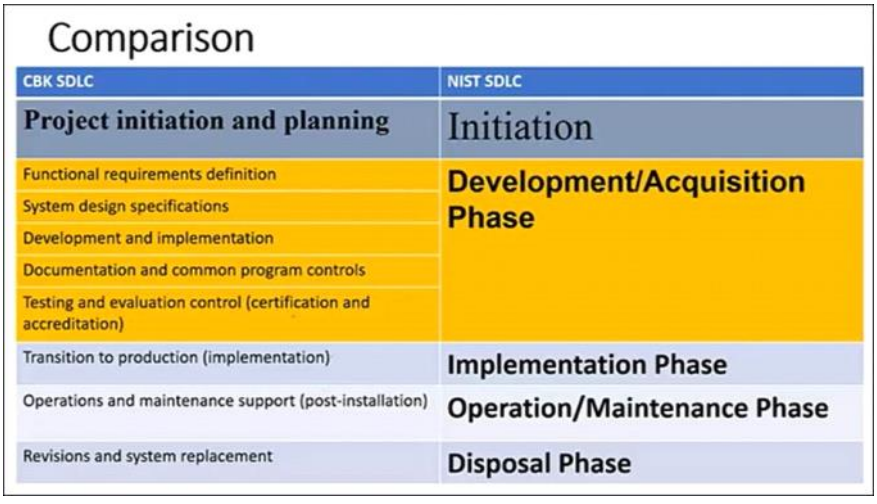


- **Initiation Phase**
  - **All Key stockholders have a common understanding of security implications.**
  - **Contingency Planning most important in the Initiation phase**
  - Security planning should begin in the initiation phase with the identification of key security roles to be carried out in the development

of the system.
- Early planning and awareness will result in saving in costs and staff time through proper risk management planning.
- The need for a system is expressed and the system purpose and high-level requirements are documented.

- **Development/Acquisition Phase**
  - **Functional Security testing is MOST Critical in the Development/ Acquisition Phase**
  - During the phase, The system is designed, purchased, programmed, developed, or otherwise constructed. This phase often consists of other defined cycles, such as the system development cycle of the acquisition cycle.
  - The organization should analyze security requirements, perform functional and security testing.
  - Prepare initial documents for system certification and accreditation, and design the security architecture.
- **Implementation/Assessment**
  - Configure and test the security feature
  - Design reviews and system should be performed before placing the system into operation
- **Operation/Maintenance Phase**
  - **Authentication of user and processes happens in the Operation/Maintenance phase**
  - System & products are in place and operating, enhancements and /or modifications to the system are developed and tested, and hardware and software components are added or replaced
  - Configuration management (CM) and Control activities should be conducted to document any proposed or actual changes in the security plan of the system.
  - Documenting information system change and assessing the potential impact of these changes on the security of a system.
- **Disposal Phase/Sunset Phase**
  - Plans are developed for discarding system information, hardware, and software and making the transition to a new system.

Comparing CBK SDLC with NIST SDLC



For questions and more clarity refer SDLC Concept Exam Dose Link

- **Assess Software Assurance Security**
- Software Assurance
  - Software Assurance can be defined as having a high level of confidence that software is free from vulnerabilities, either intentionally designed into the software or accidentally inserted at any time during its lifecycle, and that it functions in the intended manner.
  - Software Assurance in the Phases of Acquiring Software
    - To ensure the integrity of business operations and key assets within a critical infrastructure that relies on software, acquired software must be reliable and secure just as if it was developed in-house.
- These are the major phases:
  - **Planning Phase**
    - Needs determination for acquiring software services or products, identifying potential alternative software approaches, and identifying risks associated with those alternatives. This includes the following:
      - Developing software requirements to be included in work statements
      - Creating an acquisition strategy and/or plan that includes identifying risks associated with various software acquisition strategies
      - Developing evaluation criteria and an evaluation plan
  - **Contracting Phase**
    - This phase includes three major activities:
      - Creating and issuing the solicitation or request for proposal (RFP) with a work statement, instructions to potential respondents of RFP, terms, and conditions, including conditions for acceptance, prequalification considerations, and certifications.
      - Evaluating supplier proposals submitted in response to the solicitation or RFP.
      - Finalizing contract negotiation to include changes in terms and conditions and awarding the contract.
    - Software risks should be addressed and mitigated through terms and conditions, certifications, evaluation factors for award, and risk mitigation requirements in the work statement
  - **Monitoring and Acceptance Phase**
    - This phase involves monitoring the supplier's work and accepting the final service or product delivered under a contract.
    - This phase includes three major activities as well:
      - Establishing and consenting to the contract work schedule
      - Implementing change or configuration control procedures

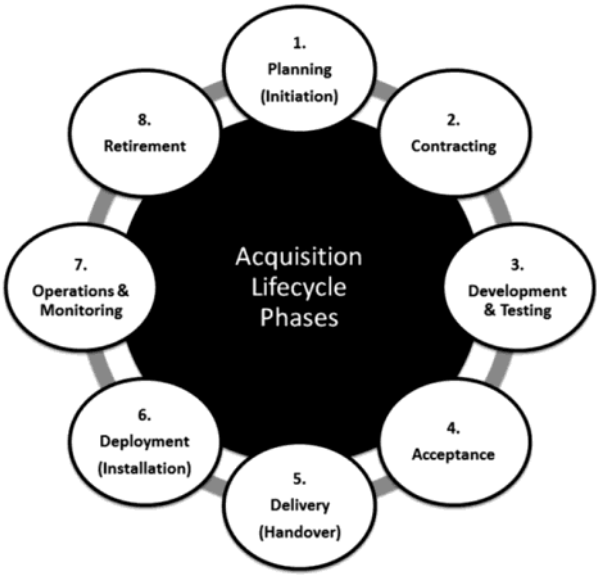
        - Reviewing and accepting software deliverables
    - During the monitoring and acceptance phase, software risk management and assurance case deliverables must be evaluated to determine compliance in accepted risk mitigation strategies as stated in the requirements of the contract.
- **Follow-on**
    - This phase involves maintaining the software. This process is sometimes called sustainment. This phase includes two major activities:
        - Sustainment, which includes risk management, assurance case management, and change management
        - Disposal or decommissioning
    - During the follow-on phase, software risks must be managed through continued analysis of the assurance case and should be adjusted to mitigate changing risks

- **Security Considerations in Managing COTS Software**
    - COTS software is generally a more attractive target than custom code.
    - It Is Difficult to Verify the Security of COTS Products
        - COTS products are generally black boxes to their customers.
        - black-box testing is possible, as will be discussed later, but this can be very difficult for COTS customers.
        - It can be very difficult to test these often very large systems exhaustively in their complex operating context.
    - Mitigating risk
        - Identify the COTS Components
        - Understand What Counts
        - Understand How Things Connect
        - Operate a Secure Computing Infrastructure
        - Control Access
        - Ask the Vendor
        - Engage with the User Community and Security Community
        - Engage with Experts
        - Test the Software
        - Look for Certification
        - Pay Attention to Updates
        - Understand How the COTS Software Operates—To the Extent that You Can
        - Audit
        - Prepare in Advance for Failure
    - **Some Summary Observations on Buying and Using COTS Code**
        - Adopt a broad definition of COTS and learn to recognize it when you see it.
        - Consider security as part of the purchase process.
        - Research security from all available directions.
        - Assume the software is insecure until proven otherwise.
        - Consider the environment in which the COTS system will operate; what may be secure in one instance, may not be in another.
        - Protect yourself—legally and physically. Request guarantees from COTS vendors and, if possible, indemnification. Where testing or risk indicate, wrap the software with a package that validates input and/or output. Protect against failure.
        - Track changes and adapt the security strategy accordingly.
        - Build-in monitoring systems to detect a failure.

- **For questions and more clarity refer COTS Coffee Dose** Link
- Url =https://us-cert.cisa.gov/bsi/articles/best-practices/legacy-systems/security-considerations-in-managing-cots-software

Software Acquisition Lifecycle

The supply chain can be broken down into distinct phases within the acquisition lifecycle. These phases are namely, planning, contracting, development & testing, acceptance, deployment, operations & monitoring, transitioning, and retirement as depicted in Figure 8.2.

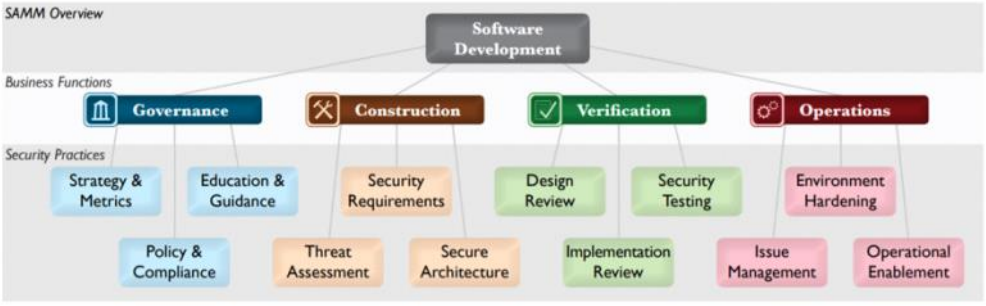


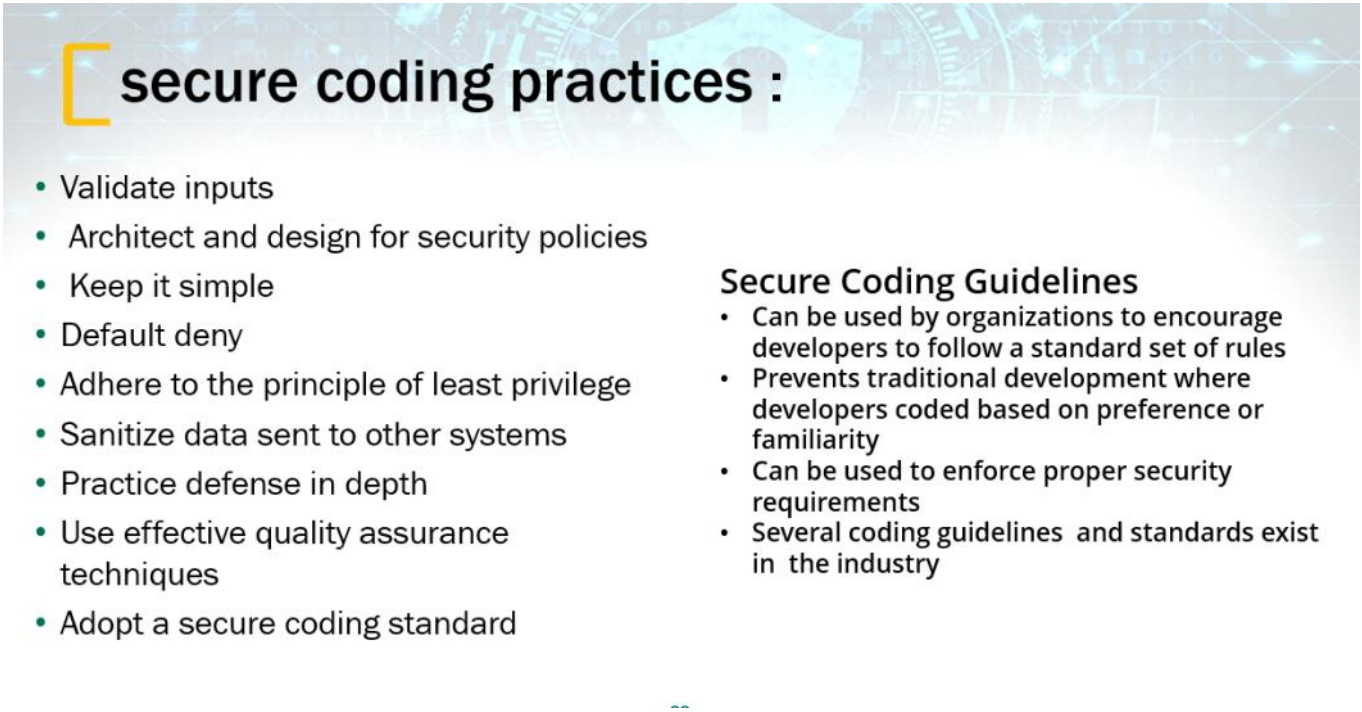
- ***Planning*** involves conducting an initial risk assessment to determine the functional and assurance needs followed by the development of an acquisition strategy and/or plan. The plan should not only cover the requirements (needs that were determined as part of the initial risk assessment) to be met, but also specify evaluation criteria. Evaluation criteria must include the following categories –organization, People, Processes, and Technology.
- ***Contracting*** involves the issuance of an advertisement to source suppliers, evaluation of the supplier and their responses (proposal to meet requirements), contract negotiations, supplier selection, and contract award.
- *Development & Testing* involves the implementation of reliable, resilient, and recoverable code and attestation of security controls.
- ***Acceptance*** involves the definition of acceptance criteria, verification and validation activities including independent third-party testing, issuance of the purchase order (PO), the acquirer acceptance, and contract closure.
- ***Delivery*** involves the establishment of code escrow agreements, if needed, and communicating and attesting compliance with export control and federal trade data regulations, in addition to secure transfer. The secure transfer involves the protection of the delivery channels and processes so that the software is not only free of being tampered with but authentic in its origin when it is transitioned from one supplier to another or the acquirer.
- ***Deployment*** involves the installation and configuration of the software with the least privileged and secure defaults, besides configuring perimeter defense controls and validating the components of integrated systems once deployed.
- ***Operations & Monitoring*** involves the enforcement of the contract work schedule and follow-on post-deployment support, the establishment of change or configuration control procedures as part of assurance case management (transitioning to operations), performing runtime integrity checks, patching and upgrades, implementing termination access controls, checking custom code extensions and continuous monitoring including the detection and handling of security incidents.
- ***Retirement*** involves the activities that help mitigate or avoid information disclosure risks. Decommission of the software, including termination access controls and disposal of the data are carried on in this phase.

Risks to software arise from threats that are introduced in one or more ways into the supply chain, either during the development and testing phases or during its deployment, operations, and maintenance phases. Some of these ways are listed below:

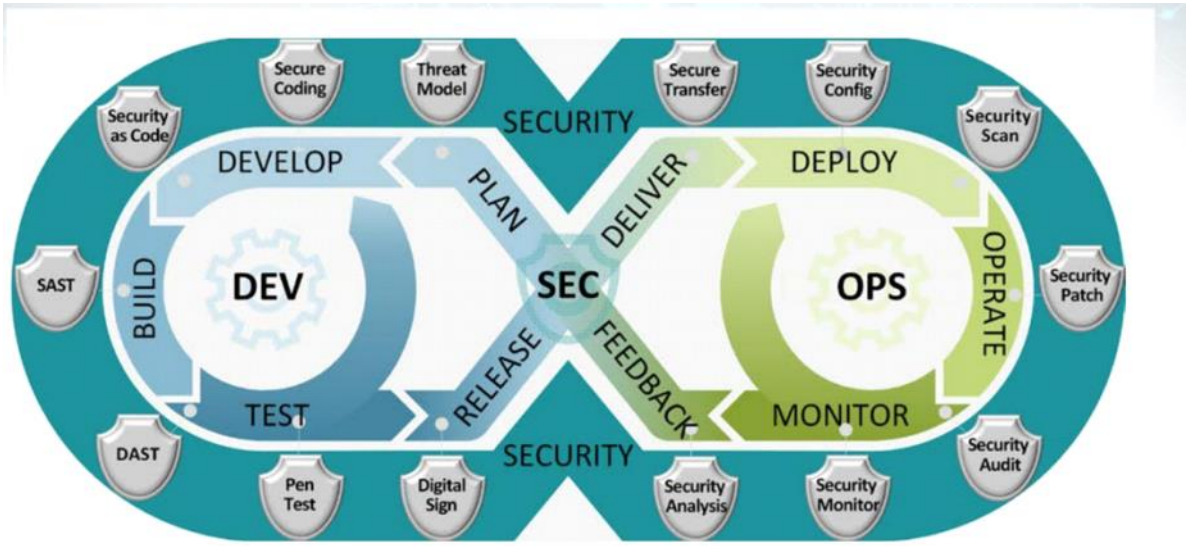
- Insufficient validation and sourcing of suppliers.
- Contractual language does not take into account security requirements.
- Unintentional design defects and coding errors that allow for exploitable vulnerabilities.
- Introduction of malicious logic code by unauthorized parties after the software is developed.
- Failure in logistics management resulting in the distribution of code without adequate access control checks.
- Improper code publishing processes that do not assure the authenticity of code origin.
- Inadequate configuration controls leading to insecure installation and deployment.
- Failure in vulnerability and patch management processes that introduces risk during operations.
- Inadequately trained personnel that fail to communicate assurance requirements to the supplier and/or attest the existence and effectiveness of technical controls in the acquired software

- 





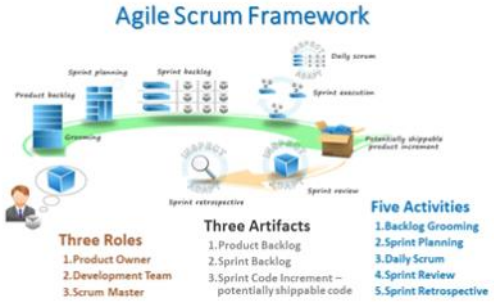
- **NOTE:-**
  - Accreditation -- Final Stage
  - Certification -- involve in newly purchase product, Testing the controls
  - Assurance -- Audit
  - Acceptance --- code met the customer requirement, UAT.
- **DevOps**
  - DevOps is a set of software development practices that combines software development (Dev) and information technology operations (Ops) to shorten the systems development life cycle while delivering features, fixes, and updates frequently in close alignment with business objectives.
  - In broad terms, DevOps is an approach based on lean and agile principles in which business owners and the development, operations, and quality assurance departments collaborate to deliver software in a continuous manner that enables the business to more quickly seize market opportunities and reduce the time to include customer feedback
  - **The primary concern introduced in DevOps that was not there in the Traditional methodology - Separation of Duties and Conflict of Interest.**
  - The core common principles that DevOps is usually made up of some of the following:
    - Develop and test against production-like systems
      - The goal is to allow development, operations, and quality assurance teams to develop and test against systems that behave just like the actual production system
    - Deploy with repeatable, reliable processes
      - This principle allows development and operations to support the development process through to production.
    - Monitor and validate operational quality
      - This principle moves the concept of monitoring earlier in the lifecycle by requiring that automated testing be done early and often to monitor important and valuable characteristics of the application.
    - Amplify feedback loops
      - This principle calls for organizations to create communication channels that allow all stakeholders to access and act on feedback in a very efficient manner.
  - Integrated Product and Process Development (IPPD)
    - Integrated Product and Process Development (IPPD) is a management technique that simultaneously integrates all essential acquisition activities through the use of multidisciplinary teams to optimize the design, manufacturing, and supportability processes
- Integrated Product Team
  - One of the key IPPD tenets is multidisciplinary teamwork through Integrated Product Teams (IPTs).
  - The purpose of IPTs is to make team decisions based on timely input from the entire team (e.g., program management, engineering, manufacturing, test, logistics, financial management, procurement, and contract administration) including customers and suppliers
- **DevSecOps**
  - Instead of security continuing to exist as a standalone, isolated entity, Secure DevOps aims to integrate security into the development processes from inception
  - The Secure DevOps approach enables developers to learn more about what they are developing and how it can be exploited.
  - Secure DevOps proactively focuses on surviving by providing reliable software with a reduced attack surface

- 

  - Capability Maturity Model (CMM)
    - The Capability Maturity Model (CMM) is a methodology used to develop and refine an organization's software development process. The model describes a five-level evolutionary path of increasingly organized and systematically more mature processes.
- **Maturity Model**

  0- Incomplete
  1- Initial/Ad-hoc - Process is unpredictable, poorly controlled, and reactive
  2- Repeatable/ Managed - Project-oriented management but still reactive
  3- Defined /Documented - Project-oriented management but still reactive
  4- Quantitatively Managed / Measured - Managed Processes are measured and controlled
  5- Optimized - Focused on process improvement

- **Identify and apply security controls in development environments**
  - Development methodologies
    - Iteractive (interacting with customers regularly and sharing the status live Ex: Agile)
    - Non-Iteractive (meet customer and collect the requirements, develop the code and submit the customer, time taken)
  - Non-Iterative Model
    - Waterfall
      - Is considered to be the traditional software development methodology. It represents a phased approach to software development where each phase is completed before the next one can be started.
      - Requires a set of activities that must be performed and documented before the next phase can begin
      - The waterfall model is considered the basis for the following other methodologies, known as non-iterative models.
      - From the perspective of business in general, **the disadvantage of the waterfall model is that it demands a heavy overhead in planning and administration, and it requires patience in the early stages of a project**
    - Structured Programming Development
      - A method that programmers use to write programs allowing considerable influence on the quality of the finished products in terms of coherence, comprehensibility, freedom from faults, and security.
      - **It also allows for security to be added in a formalized, structured approach.**
    - Spiral Model
      - **Includes a Risk assessment and financial review in every phase.**
      - A very flexible model for SDLC, this works on the principle of the iterative model by repeating four phrases again and again in a spiral, allowing for improvements with each round. This model can result in a customized product.
      - A sort of nested version of the waterfall method. The development of each phase is carefully designed using the waterfall model. A distinguishing feature of the spiral model is that in each phase of the waterfall there are four sub-stages, based on the common Deming PDCA (Plan-Do-Check-Act) model, in particular, a risk assessment review (Check)
    - Cleanroom
      - In software application development, it is a method of controlling defects (bugs) in the software.
      - The goal is to write the code correctly the first time rather than trying to find the problems once they are there.
  - Iterative Model
    - This model is based on repetition and improvement. Rather than developing software-based upon completely known requirements, a set of requirements is applied, tested, and implemented.
    - Based on further requirements and suggested improvements, a new version or iterative version of the software is produced until the final product is complete.
    - Breaking down a large application into smaller chunks, code is designed, developed, and tested in a repeated cycle.
    - **The advantage of this model is that a basic working version can be produced early but its disadvantage is that it can quickly eat up your resources because of repetition of the process.**
    - Iterative models allow for successive refinements of requirements, design, and coding.
    - Iterative models also make it very difficult to ensure that security provisions are still valid in a changing environment
    - Prototyping
      - Prototyping was formally introduced in the early 1980s to combat the perceived weaknesses of the waterfall model with regard to the speed of development.
      - The objective is to build a simplified version (prototype) of the application, release it for review, and use the feedback

from the users' review to build a second, better version
    - Modified Prototype Model (MPM)
        - Ideal for Web application development
        - A refined form of the above prototyping methodology that is ideal for web application development, allows for the basic functionality of a desired system or component to be formally deployed in a quick time
        - frame.
    - Rapid Application Development (RAD)
        - Strict time limit to develop applications. Quality is a concern, poor design
        - The goal is to produce quality code quickly. While this sounds attractive, it must be handled properly because the quick development process may be a disadvantage if decisions are made so rapidly that it leads to poor design.
    - Joint Analysis Development (JAD)
        - Originally invented to enhance the development of large mainframe systems. Recently, JAD facilitation techniques have become an integral part of RAD, web development, and other methods.
        - **It is a management process that helps developers to work directly with users to develop a working application**
    - Exploratory Model
        - Assumption is made as to how the system might work
        - This is a set of requirements built with what is currently available. Assumptions are made as to how the system might work, and further insights and suggestions are combined to create a usable system. Because of the lack of structure, security requirements may take second place to enhancements, which may be added on an ad hoc basis.
- Agile Scrum
    - The agile model relies on customer interaction and feedback. It breaks the product into cycles and delivers a working product as an ongoing release with incremental changes from the previous cycle. The product is tested with each iteration
    - Roles in Agile Model
        - Product owner -- Shares the input
        - Scrum Master --- Define the Principle
        - Development Team -- Develop the application
    - It is Lean and Customer Focused
    - It Acknowledges the facts that customers' needs will change over time
    - It focuses on team collaboration, customer involvement, and continuous delivery.
    - Allows product features to be added, removed, modified at clearly defined points.
        - Change points happen after each sprint fixed duration development interval.
    - 

- V Model
    - The V-model emphasizes verification and validation at each phase and testing to take place throughout the project
- Extreme Programming
    - Despite the name, extreme programming is an attempt to use a structured approach to software development, relying on subprojects of limited and defined scope and developers always working in pairs.
- Reuse Model
    - An application is built from already existing and tested components.
    - The reuse model is best suited for projects using object-oriented development because objects can be created, exported, reused, or modified as required.
    - From a security perspective, the components would then be chosen based on the known effectiveness of the security characteristics.

- Database Management System (DBMS) Architecture
    - A suite of application programs that typically manage large, structured sets of persistent data.
    - Stores maintain and provide access to data using ad hoc capabilities.
    - Element of DBMS
        - The database engine itself
        - The hardware platform
        - Application software
        - Users
    - **Database Models**
        - The relationship between the data elements and provides a framework for organizing the data.
        - Any database model needs to provide the following requirements:
            - Transaction Persistence - The state of the database is the same after a transaction against the database has occurred as it was prior to the transaction, and the transaction should be durable, meaning it lasts.
    - **Fault tolerance and recovery**
        - In the event of a hardware or software failure, the data should remain in its original state without impacting the security of that data.
        - **Two types - Rollback & Shadowing**
            - Rollback
                - incomplete/invalid transactions are backout
                - Rollback recovery is when incomplete or invalid transactions can be backed out properly

        - Shadowing
            - transactions are re-applied to the previous version
            - Shadow recovery occurs when transactions are reapplied to a previous version of the database
    - Sharing by multiple users
        - The data should be available to multiple users at the same time without endangering the integrity of the data or the integrity of the database environment itself.
    - Security controls
        - Including confidentiality, integrity, availability, and others that address requirements of access controls, integrity checking, and view definitions
    - Data model is fundamental to the design, it provides a mechanism for representing the data and any correlation.
    - There are different models available
        - Hierarchical Database model
            - A hierarchical database model is a data model in which the data are organized into a tree-like structure. The data are stored as records that are connected through links. A record is a collection of fields, with each field containing only one value.
            - It's simple
            - There is no correlation
    - Network Database Management Model
        - Represents its data in the form of a network of records and sets that are related to each other
        - Records are the equivalent of rows in the relational model
        - Record types are sets of records of the same type.

- **Guidelines for Implementing Security in the SDLC Here are some guidelines you can use for implementing security in the software development lifecycle:**
    - Although the CISSP is not expected to be a programmer, be sure to familiarize yourself with programming basics and terminology to facilitate communication with developers
    - Make sure that initial security questions (ones that will impact the company) are addressed during the Concept and Initiation phase of the SDLC.
    - In the Requirements phase of the SDLC, use a variety of methods to gather requirements, especially from end-users who will have to use the product
    - In the Design Phase of the SDLC, make sure that security requirements are properly articulated and included in the design
    - In the Build phase of the SDLC, make sure that developers unit test as much of their code as possible to minimize errors later on
    - In the Test phase of the SDLC, make sure that end-users not only like the product but that they are also utilized to see if they discover any additional bugs in the software.
    - During the Deploy phase of the SDLC, try a limited pilot deployment first before conducting a full-scale rollout of the product.
    - Use the Waterfall model if your project is big and complex.
    - Use the Cleanroom model if your project is highly security-oriented, and you are willing to spend the time and resources to write code correctly the first time.
    - Use an Agile model if you want to create high-quality code in a time-boxed, focused manner
    - Use the Prototyping model if the customer is not sure exactly what they want.
    - Use the Spiral model if you want to iteratively develop software while performing a risk analysis at the start of each iteration.
    - If none of the above software development models works for you, consider other models such as RAD, JAD, Exploratory, Component-based, or reuse
    - Use structured development if you want code that is easy to verify and test.
    - Use CASE tools to help automate your application development

- Relational DBMS (RDBMS)
    - The most important part of a database is RDBMS
    - A relational database management system (RDBMS) is a collection of programs and capabilities that enable IT teams and others to create, update, administer and otherwise interact with a relational database.
    - Most commercial RDBMS use Structured Query Language (SQL) to access the database, although SQL was invented after the initial development of the relational model and is not necessary for its use.
    - **Attributes corresponding columns to the table. The first information in the table is the primary key.**
    - Key Differences between Primary key and Foreign key.
        - A primary is a set of attributes/a candidate key that distinctly identifies a record in a relation. Primary keys provide the addressing mechanism within the relational model.
        - A foreign key in a table refers to the primary key of another table. The foreign key in a relational model is different from the primary key.
        - A foreign key constraint cannot be applied to the temporary tables.
- Elements of the Relation Model
    - The relational model consists of three elements:
        - Data structures that are called either tables or relations
        - Integrity rules on allowable values and combinations of values in tables
        - Data manipulation agents that provide the relational mathematical basis and an assignment operator
- Attributes of the Table **(EXAM IMP)**
    - Attributes
        - Attributes are columns in a table.
        - Attributes are unordered left to right, and thus, are referenced by name and not by position.
    - Tuple
        - A row in the table is referred to as a tuple.
        - A table has rows and columns, where rows represent records and columns represent the attributes.
        - Tuple – A single row of a table, which contains a single record for that relation is called a tuple.
        - Relation instance – A finite set of tuples in the relational database system represents relation instance
    - **Primary Key**
        - The primary key is an attribute or set of attributes that uniquely identifies a specific instance of an entity.
        - Each table in a database must have a primary key that is unique to that table.

        - Any key that could be a primary key is called a candidate key. The candidate key is an attribute that is a unique identifier within a given relational table.
        - One of the candidate keys is chosen to be the primary key and then the others can be referred to as alternate keys.
    - **Foreign Key**
        - When the primary key of one relation, or table, is used as an attribute in another table, it is referred to as the foreign key in the other table.
        - The foreign key is a primary key in another table that is used to provide a relationship in another table.
- The link between the foreign and primary keys represents the relationships between tuples.
- Foreign keys also provide a method for maintaining referential integrity in the data and for navigating between different instances of an entity.
- Integrity Constraints in Relational Databases
    - To solve the problems of concurrency and security within a database, the database must provide some integrity
    - Entity integrity and referential integrity are important considerations in relational database environments.
    - There are two integrity rules of the relational model that are very important to always address and ensure. These are entity integrity and referential integrity.
    - The two rules apply to every relational model and focus on the primary and foreign keys
        - **Entity Integrity**
            - No Primary key attribute can have a null value.
            - Primary Key Value must be unique
            - Entity integrity means that the tuple must have a unique and non-null value in the primary key. This guarantees that the tuple is uniquely identified by the primary key value.
        - **Referential Integrity**
            - Referential integrity states that for any foreign key value, the referenced relation must have a tuple with the same value for its primary key.
            - In other words, for every foreign key value, there must be a valid relation back as a primary key somewhere else in another table.
            - When there is a relationship between two entities, those two entities need to exists
            - No record can contain a reference to the key of a non-existing record
            - **Referential integrity is based on Entity Integrity**
            - When referential integrity is designed, it can be set up to either delete all child records when the parent record is deleted or to disallow the delete operation of a customer (parent record) who has orders (child records), unless all of the child order records are deleted first.
    - Two subjects trying to access the same database, integrity is maintained
    - The integrity rule applies to database refers to Clark & Wilson Integrity rules, Clark & Wilson subject requires access to object with access rights
    - **DBMS follows Clark/Wilson - Subject access to Object**
- Effects of Concurrency
    - Database performs many transactions concurrently, Double update occurs when two programs access the same element of the DB simultaneously
    - Two processes waiting for each other to release the resources
    - **Concurrent transactions are not a risk but results can be a threat if transactions are not resolved properly.**
        - Denial of Service
        - Loss of Data integrity
- Structured Query Language (SQL)
    - Structured Query Language (SQL), which provides a way for users to issue commands against the database
    - SQL is a domain-specific language used in programming and designed for managing data held in a relational database management system (RDBMS) or for stream processing in a relational data stream management system (RDSMS).
    - It is particularly useful in handling structured data where there are relations between different entities/variables of the data.
    - **Implements DAC using GRANT & REVOKE Commands**
    - There are main components of a database using SQL:
        - **Schemas:** Describes the structure of the database, including any access controls limiting how the users will view the information contained in the tables.
        - **Tables:** The columns and rows of the data are contained in tables.
        - **Views:** Defines what information a user can view in the tables. The view can be customized so that an entire table may be visible or a user may be limited to only being able to see just a row or a column.
    - SQL Sublanguages
        - SQL consists of these three sublanguages:
            - **Data Definition Language (DDL)**
                - is used to create databases, tables, views, and keys specifying the links between tables.
                - it is administrative in nature, users of SQL rarely use DDL commands as they should be restricted to database administrators.
            - **Data Manipulation Language (DML)**
                - used to query and extract data, insert new records, delete old records, and update existing records.
            - **Data Control Language (DCL) -- Centralized Access Control Called an NDAC**
                - System and database administrators utilize Data Control Language (DCL) to control access to data.
                - It provides the security control aspects of SQL and should be the security professional's area of concern.
                - These are some of the DCL commands:
                    - COMMIT: Saves work that has been done
                    - SAVEPOINT: Identifies a location in a transaction to which you can later roll back if required
                    - ROLLBACK: Restores the database to its state at the last COMMIT _
                    - SET TRANSACTION: Changes transaction options such as what rollback segment to use

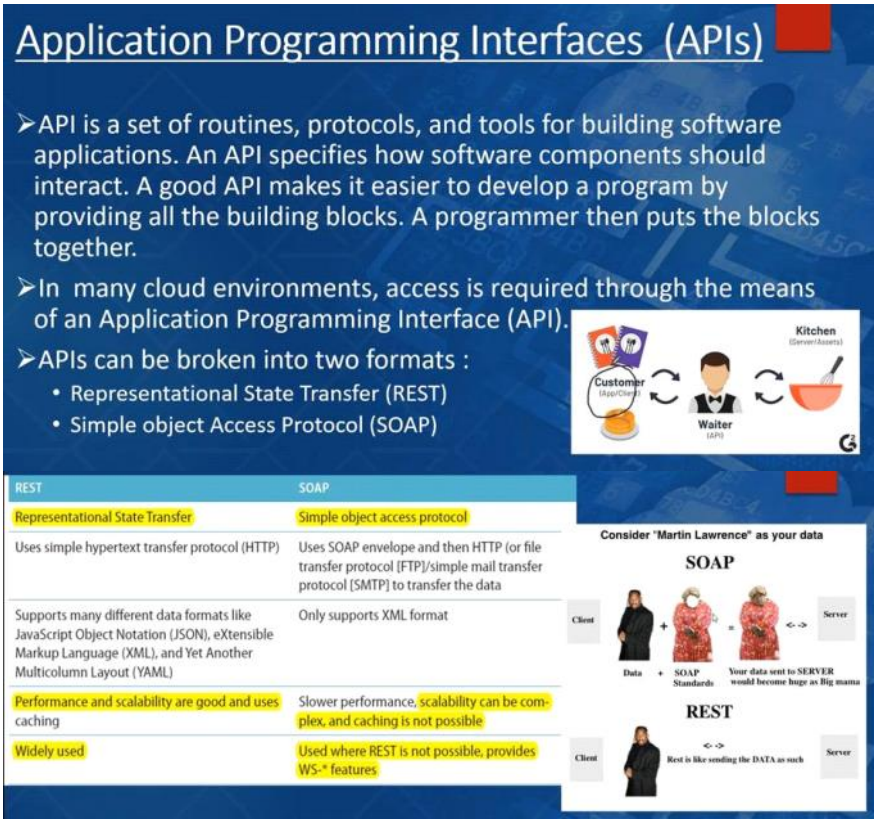
- Object-Oriented (OO) Database Model
    - The object-oriented (OO) database model is one of the newest database models. It is very similar to OOP languages, and as such, the OO database model stores data as objects.

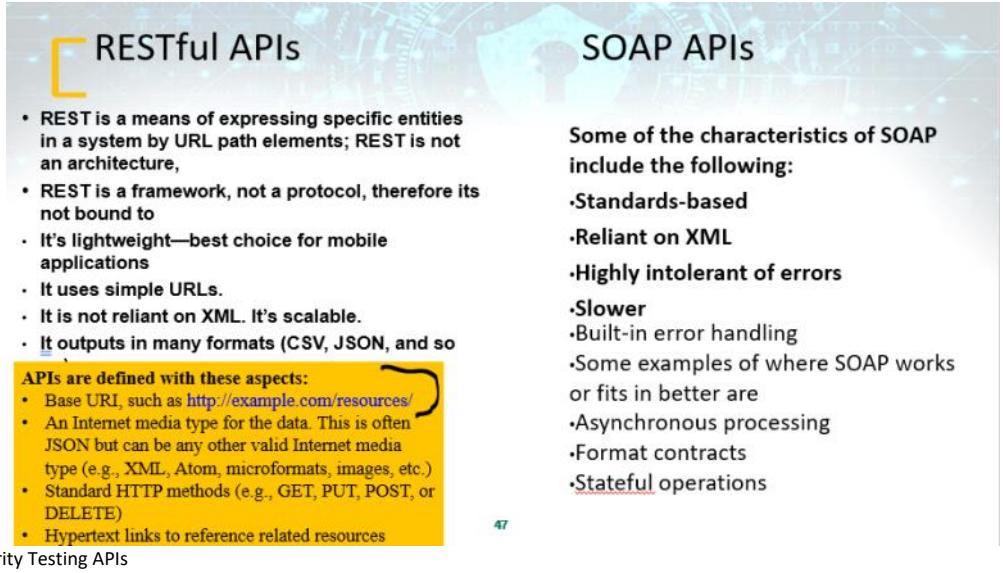
- The objects are a collection of public and private data elements and the set of operations that can be executed on those data elements.

- Database Interface Languages
  - Database Interface Languages and some of them include the following
    - Open Database Connectivity (ODBC)
      - It was developed and maintained by Microsoft.
      - It is considered to be an API that is used to provide a connection between applications and databases.
    - Java Database Connectivity (JDBC)
      - JDBC is Sun Microsystems' technology.
      - It is an API used to connect Java programs to database environments.
      - It is used to connect a Java program to a database either directly or also by connecting through ODBC, depending on whether the database vendor has created the necessary drivers for Java.
    - Extensible Markup Language (XML)
      - XML is referred to as a markup language that is used to store and transport data across networks.
    - Object Linking and Embedding Database (OLE DB)
      - Object Linking and Embedding (OLE) is a Microsoft technology that allows an object, such as an Excel spreadsheet, to be embedded or linked to the inside of another object, such as a Word document.
    - ActiveX Data Objects (ADO)

- **Application Programming Interfaces (APIs)**
  - Several types of APIs can be used to connect the end-user applications to the back-end databases stored on servers.
  - There are several security issues related to API technologies and capabilities that the security professional must be aware of.
  - These potential vulnerabilities need to be addressed through authentication of users, authorizations of users, encryption, and protection of the data from unauthorized entry, accountability and auditing, and availability and redundancy of current data.
  - Are the connectors for the Internet of Things (IoT), allowing our devices to speak to each other.
  - **Access** - Who is allowed to access data and services
  - **Request** - what data or services can be asked for (e.g. If I give you an address can you tell me how to get there)
    - Methods - Type of questions you can ask, assuming you have access
    - Parameter - Additional details you can include in the questions
  - **Response** - the data or service for your request
  - API Security issues include:
    - Authentication of users
    - Authorization of users
    - Access control
    - Encryption
    - Protection of the data from unauthorized entry, accountability, and auditing
    - Availability of current data
  - Types of API
    - Restful API
      - A RESTful API is an application program interface (API) that uses HTTP requests to GET, PUT, POST, and DELETE data.
      - REST is a framework, not a protocol.
      - Its lightweight, simple URLs, not reliant on XML It's Scalable and outputs in any format such as CSV, JSON, YAML, XML.
      - It's Efficient which means it uses smaller massage than XML
      - Some examples of situations where REST work well are
        - When bandwidth is limited
        - When stateless operations are used
        - When caching is needed
      - REST-Based API Security Recommendations
        - Employ the same security mechanisms for your APIs as any web application your organization deploys.
        - Do not create and implement your security solutions.
        - Unless your API is free, read-only public API, do not use single key-based authentication
        - Do not pass unencrypted static Keys
        - Use HMAC
    - SOAP
      - Provide more security than REST
      - SOAP API is designed to expose certain aspects of an application's business logic on a server, and SOAP uses a service interface to do this while REST uses URIs. REST APIs access a resource for data (a URI);
      - SOAP APIs perform an operation
      - Some of the characteristics of SOAP include the following :
        - Standards-based
        - Reliant on XML
        - Highly intolerant of errors
        - Slower
        - Built-in error handling
      - Some Examples of where SOAP works or fits in better are
        - Asynchronous Processing
        - Format contracts
        - Stateful operations



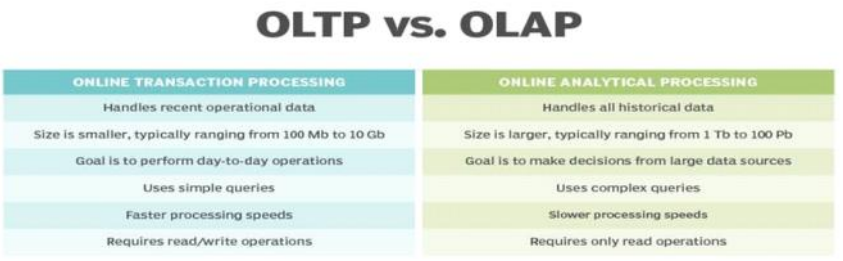


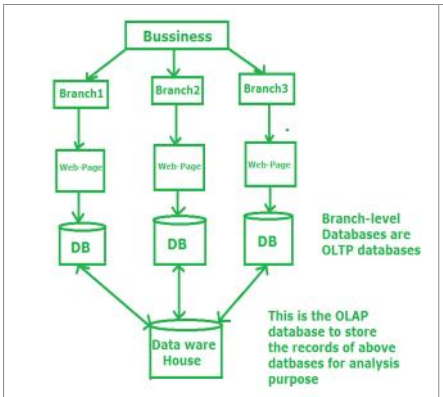


- Security Testing APIs
  - Testing should be performed to validate the behavior of an API and verify the security.
  - A variety of test methods should be employed to see how APIs handle unexpected results.
  - Fuzz testing an API can discover susceptibility to denial of service, program crashes, failing code assertions, and potential memory leaks
  - Smart fuzzing is used during white-box testing to lower the amount of data and speed up the fuzzing process
  - Monkey testing is also a form of testing that intentionally focuses on breaking the software.
    - Smart monkey testing is a particularly useful API testing technique because it is monkey testing based on knowledge of the system
  - Modules, Libraries, and Packages
    - Module, library, and package are often used interchangeably to refer to the concept of a piece of included software
    - There are several things to consider with libraries and packages
      - Verification of the security software supply chain that produced the software
      - Verification of trustworthiness of the vendor
      - Integrity and authenticity of all libraries and packages
      - Black-box security assessment of libraries and packages
      - Source security scanning and testing if source code is available

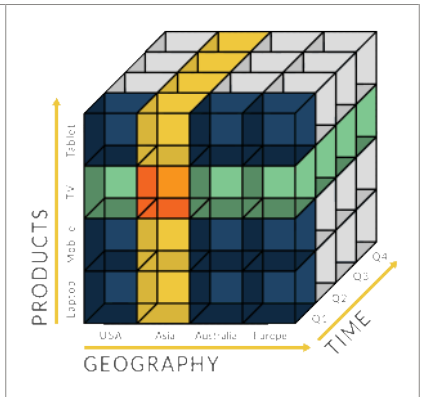
- Metadata
  - Metadata - Primary concern is Privacy data security

- Metadata is useful because it provides
  - Valuable information about the unseen relationships between data
  - The ability to correlate data that were previously considered unrelated
  - The keys to unlocking critical or highly important data inside the data warehouse.
- Data contained in a data warehouse is typically accessed through front-end analysis tools such as online analytical processing (OLAP), data mining, or knowledge discovery in databases (KDD) methods
  - Online Analytical Processing (OLAP)
    - **A function which we perform on Stored Database**
    - Enables users to selectively extract and view data.
    - OLAP data is stored in a multidimensional database, Record Attributes are seen in separate dimensions
    - Record attribute is seen as separate dimensions
    - OLAP Software can locate the intersections of the dimensions
    - Handles all historical data --- Uses complex queries
    - Size is larger, typically 1Tb to 100Pb --- Slower processing speeds
    - Goal is to make decisions from large data sources --- Requires read-only operations
    - Security Concerns
      - Privacy concerns
      - Amount of detail can divulge and profile a customer
      - Risk is increased if this information's are stored in a web server
      - Can produce inaccurate data relationships
      - Control Measures
        - Use Integrity models - Clark Wilson or BIBA
  - Online Transaction Processing (OLTP)
    - It is used when databases are clustered to provide fault tolerance and higher performance
    - Main goal is to ensure that transactions either happen or don't happen at all
    - It can load balance incoming requests if necessary
    - It provides integrity by ACID test
    - Handles all recent operational data - Uses simple queries
    - Size is smaller, typically range from 100Mb to Gb - Faster processing speeds
    - Goal is to perform day to day operations Requires Read Write operations
    - Security Concerns
      - Security concerns for OLTP system - Two primary concerns
        - Concurrency control - ensures two subjects can't modify the subject.
        - Atomicity - all critical steps must be completed for integrity.

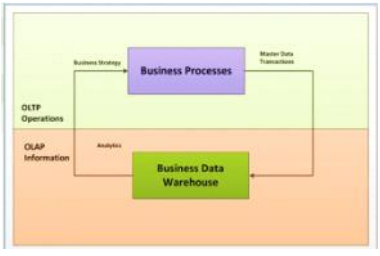


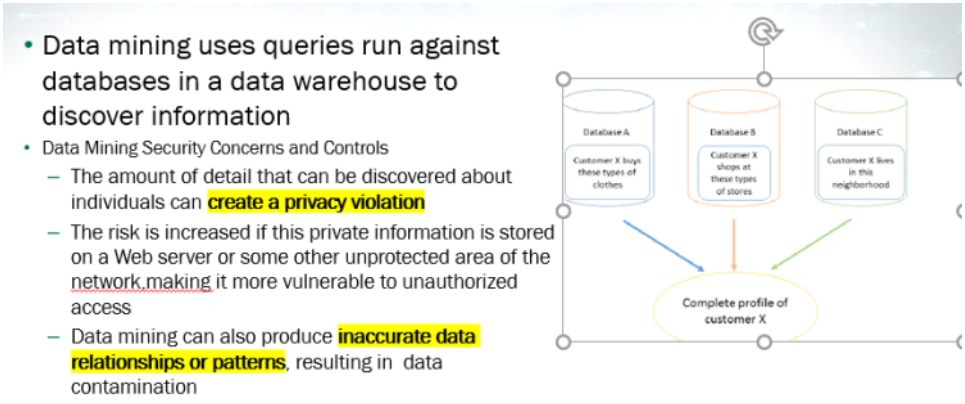


Example of OLTP database



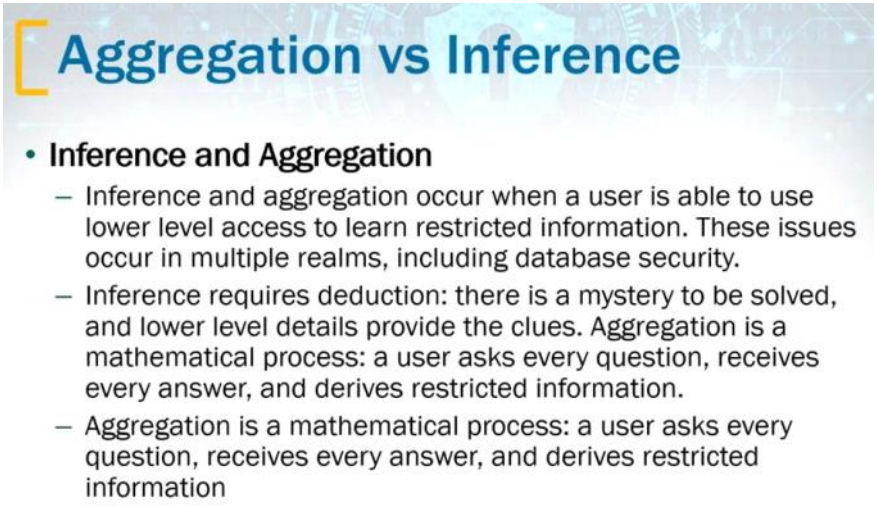
Example of OLAP database

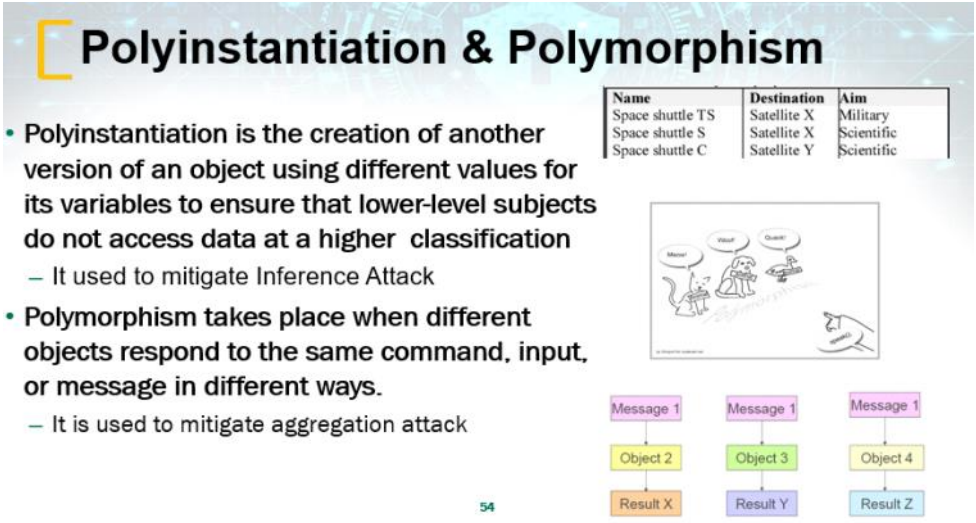


- Data Mining
  - Data Mining uses queries run against a database in a data warehouse to discover information.
  - Data mining is used to reveal hidden relationships, patterns, correlations, and trends in the data warehouse, which is a large repository purposely set up for data mining.
  - Data Mining Security Concerns and Controls
    - The amount of details that can be discovered about individuals can create a privacy violation.
    - The risk increased if this private information is stored on a web server or some other unprotected area of the network, making it more vulnerable to unauthorized access.
    - Data Mining can also produce inaccurate data relationships or patterns, resulting in data communication
  - Security professionals and stakeholders may be able to use data-mining tools to mine security intelligence to drive better controls and address vulnerabilities more efficiently and cost-effectively.
  - One possibly positive security element of data mining is to use the same mining tools to review audit logs to determine intelligence related to events and incidents
  - 

- Data Analytics
  - The expected next step from data mining is data analytics. Data analytics uses data mined from different sources to predict an outcome.
  - Data Analytics Security Concerns and Controls
    - Attacks will have many farther-reaching consequences to reputation, legal liability and may even cause a company financial ruin.
    - Moving the data to the cloud could inadvertently remove permission or confidentiality restrictions on the data
    - Privacy is at greater risk than ever before.
    - Implement privacy best practices directly into the logic of your analytics programs
    - Implement transparency and accountability in your data analytics.
- Database Vulnerabilities and Threats
  - One of the primary concerns related to any database environment is the confidentiality of sensitive information.
  - We also need to be concerned with availability and integrity controls. Especially with databases, integrity and accuracy of the content are very important.
    - **Aggregation:** The ability to combine non-sensitive data from separate sources to create sensitive information is referred to as aggregation. **Polymorphism is used to control**
    - **Bypass Attacks:** User attempt to bypass controls at the front end of the database application to access information
    - **Compromising database views used for access control:** A view restricts the data a user can see or request from a database, one of the threats is a user may try to access restricted views or modify an existing view. Another problem with view-based access control is the difficulty in verifying how the software performs the view processing.
    - **Concurrency:** When actions or processes run at the same time, they are said to be concurrent, Problems with concurrency include running processes that use old data updates that are inconsistent or having a deadlock occur.
    - **Data contamination:** The corruption of data integrity by input data errors or erroneous or inconsistent processing. This can occur in any environment including files, reports, or database environments.
    - **Deadlocking:** Occurs when two users try to access information at the same time and both are denied, In a database, a deadlocking occurs when two user processes have locks on separate objects and each process is trying to acquire a lock on an object that the other process has already locked.
    - **Inference:** Ability to deduce sensitive or restricted information from observing available information.

      Polyinstantiation is used to control
- **Interception of Data:** If dial-up or some other type of remote access is allowed, the threat of interception of the session and modification of data in transit must be controlled.
- **TOCTOU : (Time-of-check _ Time-of-use) Race Condition:** TOCTOU can also occur in database environments.
  An example is when some type of malicious code or privileged access could change data between the time that a user's query was approved and the actual time the data is displayed to the user.
- The difference between race conditions and TOC/TOU attacks is subtle but important for the security professional to understand. A race condition implies that two processes will be forced to execute out of sequence, allowing the attacker to control or manipulate the outcome. While a TOC/TOU attack happens as a result of the attackers inserting themselves in between two processes as they are executing, causing a redirection of the second process in some way to control or manipulate the outcome.



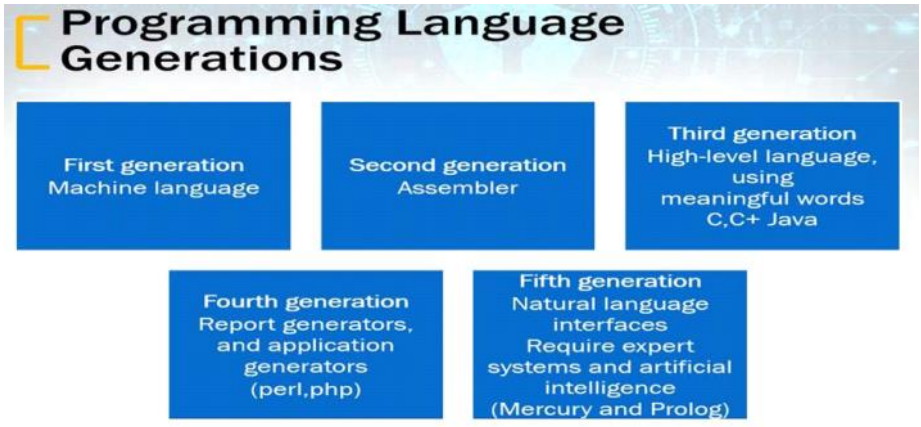


- Security Controls
  - There are two types of defensive controls in Database
    - Polyinstantiation **(EXAM IMP)**
      - Poly (multiple) Instance (versions) - Control to defense against inference attack.
      - Interactively creating two instances of the same object with different attributes; is used to stop inference attacks
      - It is the creation of another version of an object using different values for its variables to **ensure that the lower level subjects do not access data at a higher classification**
    - Polymorphism **(EXAM IMP)**
      - Poly (multiple) control to defense against aggregation attack.
  - RDBMS have a couple of tricks to ensure access controls are implemented properly:
    - **Database view**– a virtual table comprised of one or more underlying tables that restrict or aggregate data according to access control policy
    - **Polyinstantiation**– interactively creating two instances of the same object with different attributes; this is used to stop inference attacks
      - Polyinstantiation may prevent inference possibilities by creating a new version of an object by replacing the variable with other values.
  - DBMS Controls
    - The DBMS should use identification, authentication, authorization, and accountability controls as part of well-implemented access controls.
  - Lock Control (IMP)
    - Locks are used for read and write access to specific rows of data in relational systems or objects in object-oriented systems
    - Lock controls can also be very granular so that locking can be accomplished by table, row, record, or even field.
    - the ACID test, which represent the first letters of each of these terms: atomicity, consistency, isolation, and durability.
      - **Atomicity :**
        - **This is when all the steps in a transaction's execution are either all committed or all rolled back**.
        - In other words, do it all or not at all. Essentially, either all changes take effect or none do.
        - Atomicity ensures there are no incomplete or unfinished transactions in the system.
      - **Consistency :**
        - Occurs when the database is transformed from one valid state to another valid state. A transaction that is applied to a database is only allowed if it follows integrity constraints that will not affect the integrity of the database itself and its content.
        - illegal transactions are not allowed, and if an integrity constraint cannot be satisfied the transaction is rolled back to its previously valid state, and the user is informed that the transaction has failed.

        - **Isolation :**
            - Is the process guaranteeing the results of a transaction are invisible to other transactions and users of the database environment until the transaction is completed successfully.
        - **Durability :**
            - Ensures the results of a completed transaction are permanent and can survive future system and media failures once it has been applied successfully.
            - In other words, once they are done, they cannot be undone. This is similar to what is referred to as transaction persistence.
- DBMS Access Controls
    - Security controls that apply to databases can be implemented at different levels.
    - **View-based access controls (Restrict user from accessing information)**
        - View-based access control allows the database to be logically divided into pieces that allow certain sensitive data to be hidden from users that are not authorized, users.
        - Proper security controls must be put in place so that a user cannot bypass the trusted front end and directly access and manipulate the sensitive data within the database.
        - The database administrator can set up a view for each type of user and then each user can only access the view assigned to them.
- **Grant and Revoke access controls**
    - Grant and revoke controls allow users who have "grant authority" permission to grant permissions to other users.
    - In a grant and revoke system, if a user is granted permission without the grant option, the user will not be able to pass that grant authority to anyone else. This is, in a sense, **a modification of discretionary access control / If Admin manages in NDAC**
    - The revoke statement functions like the grant statement. One of the possible security characteristics of the revoke statement is its cascading effect. When the rights previously granted to a user are subsequently revoked, all similar rights are revoked for all users who may have been granted access by the newly revoked user.
- **Metadata controls**
    - Metadata can also be used to manage restricted access to sensitive information.
    - Metadata can serve as sort of a gatekeeper to enforce access rules and as a result, provide security controls.
- **Security for Object-Oriented(OO) Databases** - Protect all kind of databases
- **Data Contamination Controls**
    - To ensure the integrity of data, two types of controls can be used.
    - **Input Controls**
        - Input controls include transaction counts, hash totals, error detection, error correction, resubmission, self-check digits, and control totals
    - **Output Controls**
        - output controls may include the validation of transactions through reconciliation, physical-handling procedures, authorization controls, verification with expected results, and audit trails.
- **Content-dependent access control -** Based on the data
- **Context-dependent access control -** Based on the actions
- **Partitioning** - Dividing Database into different parts and controlling access
- **Cell Suppression -** Hiding specific cells that contain information
- Database Security controls
    - Only install or enable those components of the database system that are needed for your application
    - Place data stores and log files on a non-system partition
    - Set appropriate file system permission on a database, data stores, logs, and certificate files
    - Run database services using a dedicated unprivileged account
    - Disable command history
    - Unique certificate for each database
    - Disable or remove an unneeded account
- **Encryption in Database**
    - Full Disk Encryption - Protect against physical theft
    - File system Encryption - Encrypting volume or file directory
    - Cell-Level Encryption - Encrypt at cell level or Colum level



- Code Security Assessment
  - Synthetic Transactions
    - Measurement of availability and response times using recorded actions that emulate a specific interaction
  - Security Code
    - Review Examination of source code to verify that the proper security controls are present and work as intended
  - Static Code Analysis
    - Examination of non-running code (static) for vulnerabilities
  - Dynamic Analysis
    - Examination of running code for vulnerabilities (automated)
  - Fuzzing
    - Automated testing technique used to discover coding errors and security loopholes by inputting invalid, unexpected, or semi-random data, called fuzz, and monitoring the application response
  - Use Case
    - Positive testing determines if the application works as expected [use case] Negative testing ensures that the application can gracefully (and securely) handle invalid input or unexpected behavior [misuse case]
  - Testing should be performed to validate the behavior of an API and verify the security
  - A variety of test methods should be employed to see how APIs handle unexpected results.
  - **Fuzz testing** an API can discover susceptibility to denial of service, program crashes, failing code assertions, and potential memory leaks
    - **Smart fuzzing** is used during white-box testing to lower the amount of data and speed up the fuzzing process
    - **Monkey testing** is also a form of testing that intentionally focuses on breaking the software.
    - **Smart monkey testing** is a particularly useful API testing technique because it is monkey testing based on knowledge of the system.
- Modules, Libraries, and Packages
  - Module, library, and package are often used interchangeably to refer to the concept of a piece of included software
  - There are several things to consider with libraries and packages
    - Verification of the security software supply chain that produced the software
    - Verification of trustworthiness of the vendor
    - Integrity and authenticity of all libraries and packages
    - Black-box security assessment of libraries and packages
    - Source security scanning and testing if source code is available
- Web Application Environment
  - The web application environment is where web applications run on a server and hosts the interface that web users use to interact with organizations.
  - As the web application environment is accessible to everyone out on the web, it becomes really important to protect the entire web application architecture and its components.
  - Most attacks are conducted at the application level due to the below reasons.
    - Designed to be widely accessible
    - Usually heavily advertised
    - Administrators turn off logging
    - Not well suited for firewalls and intrusion detection systems.
  - Web Application Threats & Protection
    - Having a particular sign-off process for web servers
    - Hardening the operating system used on such servers
    - Extending web and network vulnerability scans before deployment.
    - Deploying IDS & IPS Technology
    - Use application proxy firewall
    - Only allow access from authorized hosts or networks, and then use strong (multi-factor) user authentication.
    - Disable unnecessary documentation and libraries.
    - Remove or appropriately secure administrative interface
- Security of Software Environments
  - The objective of Information security is to make sure:
    - That the system and its resources are available when needed
    - That the integrity of the processing of the data and the data itself is ensured
    - That the confidentiality of the data is protected
- Programming Language
  - A Programming language is a set of rules telling the computer what operations to perform.



- Programming Tools/Toolsets
  - A programming tool or software development tool is a program or application that software developers use to create, debug, maintain, and also, support development efforts and applications.
  - Typically, programming tools exist such as the following:
    - Binary compatibility analysis tools, Bug databases, Build tools, Code coverage, Compilation, and linking tools
    - Debuggers, Documentation generators, Library interface generators, Integration tools, Memory debuggers
    - Revision control tools, Scripting languages, Search tools, Source code editors, Source code generation tools
    - Static code analysis tools, Unit testing tools
- Integrated Development Environments (IDEs)
  - Integrated development environments (IDEs) combine the features of many tools and capabilities into one environment for use by the developer and other stakeholders.
  - Integrated development environments are designed to maximize developer productivity by providing re-usable components with similar user interfaces.
  - Integrated development environments also present a single architecture in which all development may be done.
- Runtime
  - A runtime system is the collection of all the hardware and software components that allows an application to run on a computer system.
- Source Code **(EXAM IMP)**
  - Source Code Management
    - SCM is a business and technical process that controls the record, management, sharing, and versioning of source code
    - SCM logging tracks modifications to the code
    - SCM includes several core features:
      - Source code accountability
      - Source code integrity
      - Verification
    - Improper versioning or version management is the primary reason for fixed bugs to reappear
    - Software Configuration Management(SCM) is a process to systematically manage, organize, and control the changes in the documents, codes, and other entities during the Software Development Life Cycle.

  - Security Weaknesses at Source Code Level and Secure Coding Practices
    - Buffer Overflow **(DOMAIN 3)**
      - A buffer is a data storage area inside computer memory (stack or heap)
      - A predefined amount of data can be inserted
      - if you insert too much data it creates a buffer overflow into adjacent memory, No bounds checking is done.
      - **Stack -** Fixed (Static) Counter measure: input validation & boundary testing
      - **Heap -** Dynamic (object reuse or covert storage), Read the application and allocate the space
    - Memory Reuse
      - Memory Management involves sections of memory allocated to one process for a while, then deallocated them reallocated to another process.
      - This can include random access memory (RAM), cache, or simply hard drive space
      - The problem from a security perspective is that because residual information may remain when a section of memory is reassigned to a new process after a previous process is finished with it, that information remaining on that object may be very sensitive.
    - Executable Content/Mobile Code
      - Executable content, or mobile code, is software that is transmitted across a network from a remote source to a local system and is then executed on that local system.
      - The code can arrive in the local system as attachments to email messages, or through web pages. This can be particularly dangerous because the software that is transmitted as a result may be malicious in intent.
      - Mobile code might be called by many names such as mobile agents, mobile code, downloadable code, executable content, active capsules, remote code, dynamic email, and so on.
    - Covert Channel
      - A covert channel may be defined as a communication channel that allows processes to transfer information in such a way as to violate some security policy or requirement.
      - Sending information in an unauthorized manner using a medium in an unintended way
      - **Covert Timing Channel -** A process relays information to another by modulating its use of system resources
      - **Covert Storage Channel -** A process writes data to a storage location and another process of lower clearance reads it. A Storage Covert channel involves the direct or indirect reading of storage locations by one process and direct or indirect reading of the same storage location by another process.
    - Between the lines Attack
      - Telecom lines are tapped in to and data is falsely inserted
    - Trapdoor / Backdoor
      - A backdoor attack bypasses authorized security controls, communication channels, and other authorized means to access system information
      - The problem of trapdoors and backdoors is when they still exist once the application or system has been placed in production. Security needs to take great care in making sure trapdoor, backdoor, and maintenance hooks don't exist in production.
- Source Code Analysis Tools
  - Source code analysis tools are designed to analyze source code and, in some cases, also compiled or machine language code.
  - The software development phases are where these tools can be helpful as they can provide immediate feedback to the developers and analysts on issues that they might be introducing into the code during code development itself.
- Strengths of Source Code Analysis Tools
  - Scale well; can be run repeatedly on various software.
  - Useful for things that such tools can automatically find with high confidence, such as buffer overflows, SQL Injection flaws, etc.
  - Output is good for developers; highlights the precise source file and line numbers affected.

- Weaknesses of Source Code Analysis Tools
  - Many types of security vulnerabilities are very difficult to find automatically, such as authentication problems, access control issues, insecure use of cryptography, etc.
  - High numbers of false positives.
  - Frequently cannot find configuration issues, since they may not be represented in the code
  - Difficult to prove that an identified security issue may indeed be an actual vulnerability.
  - Difficulty analyzing code that cannot be compiled. Analysts frequently cannot compile code because they do not have the right libraries, all the compilation instructions, or all the code available.

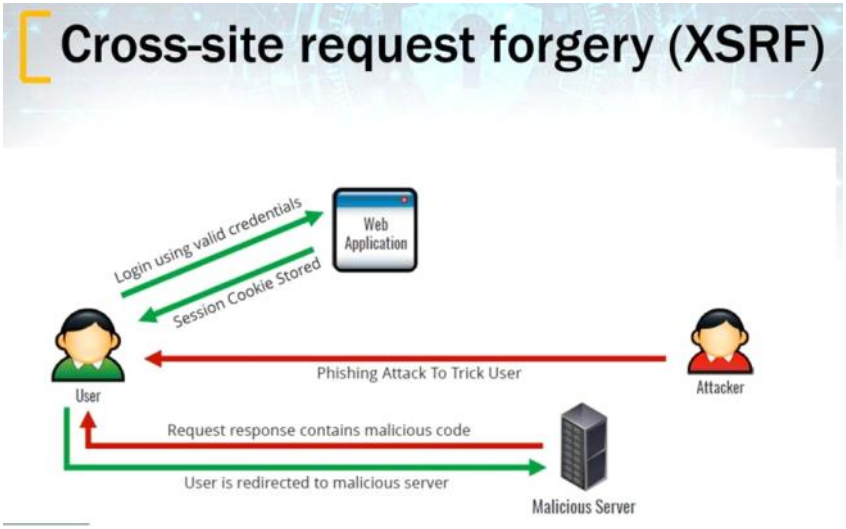
DAST (Black box testing)	SAST (White box testing)
Performed later in Software Development Lifecycle (SDLC), once the application is in a deployable state	Performed throughout the SDLC before the application is deployable
Outside-in – hacker's perspective	Inside-out – developer's perspective
Detects vulnerabilities by conducting attacks against a running instance of the application; typically only web-applications	Detects potential vulnerabilities by identifying insecure code paths without actually executing the program; supporting both web and desktop applications
Only tests program paths that are actually executed	Full code coverage, even code that is never executed
Typically finds less than SAST, but findings are more precise	Reveals a wider range of findings, but many are not exploitable (imprecise). Prioritization is difficult

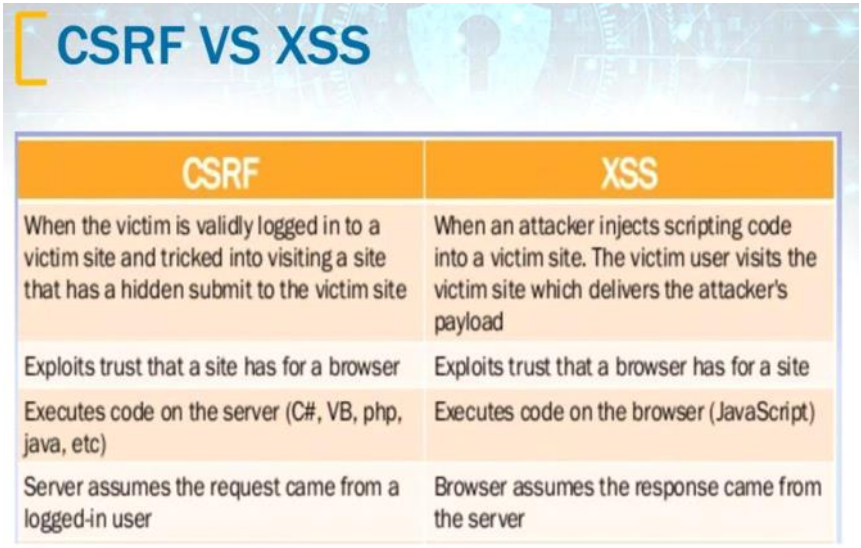
- Malicious Software (Malware)
  - Malicious software is applications that have been written to do something harmful to resources and assets that have value to the organization.
  - Malware can compromise programs and applications, but also impact data and information, **to affect its integrity and availability.**
  - Malware may use the resources of the system or architecture for nefarious purposes and in some cases exhaust CPU cycles, available processes, memory, communications links and bandwidth, open ports, disk space, mail queues, and so forth.
  - Viruses are the largest class of malware
  - Protection Techniques
    - Don't double click on attachments
    - Describe the content of attachments
    - Do not blindly use the most widely used products as a company standard
    - Disable Windows Script Host, ActiveX, VBScript, and JavaScript
    - Do not send HTML-formatted e-mail
    - Use more than one scanner, and scan everything
  - Malware Protection Tools
    - Scanner (Signature-based)
    - Heuristic Scanners
    - Activity monitors
    - Change Detection
    - Reputation Monitoring/Zero-day/Zero-hour
    - Anti-malware Policies
  - Types of Malware
    - **Worms:** A worm reproduces and spreads, just like viruses; however, worms are distinct and different from viruses although they may have similar results. **The difference is that a worm can propagate without user action.**
    - **Hoaxes:** Hoaxes are usually warnings about new viruses that do not exist. Hoaxes generally carry an instruction to the user to forward the warning to all contacts available to the user.
    - **Trojans:** Trojans, short for Trojan horse programs, are considered to be a very large class of malware. A Trojan is a program that can be seemingly useful, but it also contains something unknown that will do something malicious. In other words, it pretends to do one thing while performing another unwanted or malicious action.
    - **Social engineering:** A major component of malware success is related to the social engineering component. To recall, social engineering can be defined as using deception and intimidation to get people to do something that they shouldn't.
    - **Remote access Trojans (RATs):** RATs are programs designed to be installed, usually remotely after systems are in production and so, they are different from logic bombs and backdoors.
    - **DDoS zombies:** Distributed denial of service (DDoS) is a category of DoS attacks. DoS attacks do not attempt to destroy or corrupt data, but rather they attempt to use up a computing resource to the point where normal resources cannot proceed.
    - **Logic bombs:** Logic bombs are software programs set up to run in a dormant state until a specific condition or set of conditions exist and then activate their negative payload.
    - **Pranks:** Pranks are very much a part of the computer culture, so much so that anyone can now buy commercially produced joke packages that allow you to perform tricks on other users. There are numerous pranks available as shareware.
    - **Botnets:** A botnet is a network of automated systems or processes (robots or for short, bots) performing a specific function together, usually malicious. Botnets have greatly magnified the power and speed of malicious operations because they all work together toward achieving a malicious goal, and they have allowed for tuning and directing of operations in a way that was not possible with malicious programs in the past.
- Viruses
  - A program is written with functions and intent to copy and disperse itself without the knowledge and cooperation of the owner or user of the computer.
  - A computer virus is a malicious software program loaded onto a user's computer without the user's knowledge and performs malicious actions.
  - It can self-replicate, inserting itself onto other programs or files, infecting them in the process. Not all computer viruses are destructive though
  - There are a number of various types of viruses, such as file infectors, boot sector infectors, system infectors, email viruses, multipartite, macro viruses, and script viruses.
    - **File Infectors:** A file infector infects program or object files. System infectors that infect operating system program files are also

considered to be file infectors.
- **Boot sector infectors:** Boot sector infectors attach to or replace the master boot record, system boot record, or other boot records and blocks on physical disks.
- **System infectors:** System infector is a somewhat vague and overused term. The phrase is often used to indicate viruses that infect operating system files, or boot sectors, in such a way that the virus is called at boot time and may have control over some functions of the operating system.
- **Companion virus:** Some virus programs have been specifically designed to not physically touch the target file at all.
- **Email virus:** An email virus specifically, rather than accidentally, uses the email system to spread
- **Multipartite:** The term multipartite was originally used to indicate a virus that was able to infect both boot sectors and program files at the same time. The ability of a virus to do this was also referred to as a dual infector.
- **Macro virus:** A macro virus uses macro programming of an application and is usually said to infect office productivity tools, such as word processors.
- **Script virus:** Script viruses are generally differentiated from macro viruses in that they are usually stand-alone files that can be executed usually by an interpreter, such as Microsoft's Windows Script Host (.vbs files).
- **Polymorphic Virus:** Multiple Behaviors, after infecting it changes the signature it's difficult to be caught by the antivirus, it's also called stealth virus. A polymorphic virus is a complicated computer virus that affects data types and functions. It is a self-encrypted virus designed to avoid detection by a scanner. Upon infection, the polymorphic virus duplicates itself by creating usable, albeit slightly modified, copies of itself.

- **Control and Separation of Environments**
  - In software development, there are usually various environments, these may include, for example:
    - Development environment
    - Quality assurance environment
    - Production environment
  - To be effective, the quality assurance configuration should simulate the production environment as closely as possible. Once the testing has been completed, including the security testing, and stakeholders have accepted the application, it is moved into the production environment
  - Control measures protecting the various environments are many, but they should include physical isolation of environment, physical or temporal separation of data for each environment, access control lists, content-dependent access controls, role-based constraints, role definition stability, accountability, and separation of duties.
- **Cross-Site Scripting**
  - Cross-Site Scripting (XSS) attacks are a type of injection, in which malicious scripts are injected into otherwise benign and trusted websites. XSS attacks occur when an attacker uses a web application to send malicious code, generally in the form of a browser-side script, to a different end-user.
  - User is not required to be authenticated, sends the script. The mitigation in both cases is to never trust user input. All input must be carefully screened, preferably using a standard library that will remove or escape components of the input that, if sent back to the browser, would be interpreted as script or code (and not displayed as data).
  - Exploits trust that a browser has for a site - Executes code on Browser - Browser assumes response came from Server
  - The user is not required to be authenticated.
  - Stored XSS
    - Persistent XSS
    - Malicious Payload is stored in DB and is run by Others
  - Reflected XSS
    - Non-persistent
    - Payload is embedded in the URL
    - Victim visits malicious URL and gets exploited
  - DOM Based XSS
    - Payload executed by modifying DOM environment
    - Can be persistent / Non-Persistent
  - **Countermeasure**
    - input validation or data sanitization can also be performed to help prevent
    - Enabling content security policy as defense in depth mitigation
    - Applying context-sensitive encoding when modifying the browser document

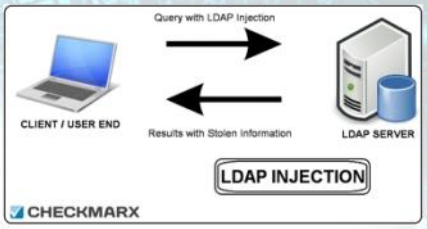
- Cross-site Request Forgery (CSRF/XSRF)
  - Cross-site request forgery (XSRF) is an attack that is similar in nature to XSS.
  - XSS sends untrusted data to a web application or browser without performing input/output validation or escaping from the code execution
  - This allows the attacker to force the victim's Brower to generate requests the vulnerable application thinks are legitimate request from the victim
  - One form of XSRF infects the victim's system with malware that stays dormant until a specific website is visited.
  - A Cross-site request forgery (CSRF) attack focuses on tricking a user to perform unwanted requests on a web application.
  - Using the Victim's valid session credentials, an attack executes malicious requests called a forged request
  - CSRF targets state-changing requests of the validated user, such as financial information, changing passwords, and updating other information.
  - Exploits trust that a site has for a browser - Executes code on the server - Server assumes request came from logged in user
  - User need to be authenticated
  - Countermeasure -- Token-based authentication, Captcha
  - **Countermeasure**
    - A unique ID is appended to each link/form on the requested page prior to being displayed to the user
    - Inserting the CSRF token in the custom HTTP request header
    - Maintaining a list of unique IDs in the user session
    - Confirmation page
    - Are you sure you want to transfer $10000?
    - Random, One-time tokens in forms
    - CSRF tokens prevent CSRF because without token, attacker cannot create a valid requests to the backend server



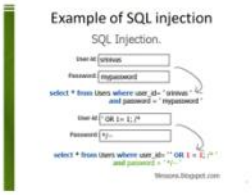


- Redirects and Forwarding
  - Unvalidated Redirects and forwards can send website users to malicious sites believing they are legitimate
  - Web applications frequently redirect and forward users to other pages and websites, and use untrusted data to determine the destination pages.
  - Without proper validation, attackers can redirect victims to phishing or malware sites, or use forwards to access unauthorized.
- SQL Injection
  - SQL injection is a code injection technique, used to attack data-driven applications, in which malicious SQL statements are inserted into an entry field for execution (e.g. to dump the database contents to the attacker).
  - A subset of unverified user input
  - SQL injection attacks use it to gain unauthorized access to an underlying database and related assets
  - Two techniques to protect your web application against SQL injection attacks
    - Perform input validation
    - Limit account privileges

Injection flaws



- Such as SQL, OS, and LDAP injection occur when untrusted data is sent to an interpreter as part of a command or query.
- The attacker's hostile data can trick the interpreter into executing unintended commands or accessing data without proper authorization
  - This is when a malicious user attempts to inject a string of some type into a field in order to manipulate the application's actions or reveal unauthorized data

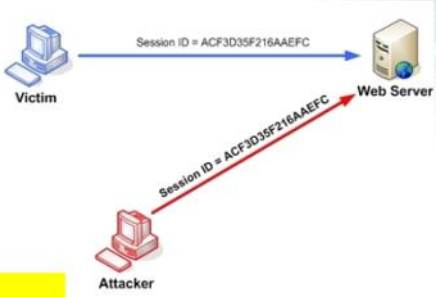


Countermeasure

- Preferred option to use safe API, which avoids the use of the interpreter entirely or provides a parameterized interface or migrate to use Object Relational Mapping Tools
- Use positive or whitelist server-side input validation
- Use LIMIT and other SQL control within queries to prevent mass disclosure of record in case of sql injection

Broken Authentication & Session Management

- This occurs when a malicious user is able to break a session and steal items like tokens, passwords, or keys. This then allows the malicious user to hijack the system.
- A third party can access to anyone's account
- Attacker compromise password, keys or authentication token Session IDs aren't rotated after successful login.



Countermeasure

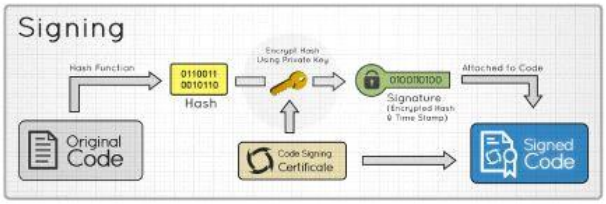
- Where possible, implement multi-factor authentication to prevent automated, credential stuffing, brute force, and stolen credential re-use attacks.
- Do not ship or deploy with any default credentials, particularly for admin users.
- Use a server-side, secure, built-in session manager that generates a new random session ID with high entropy after login

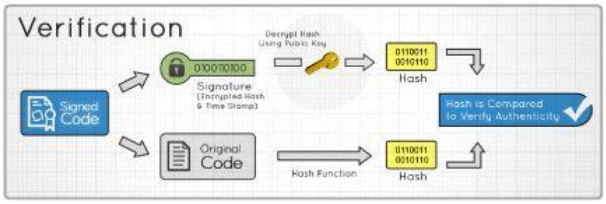
- Directory Traversal attacks
  - Trying to access a directory that there is no permission
  - AKA Path traversal attack
  - Can lead to command injection
  - Allows access to files outside **wwwroot**
  - Using the dot dot attack the user can :
    - Download files, Execute command, Steal Source code
  - How to protect yourself
    - Prefer working without user input when using file system calls
    - Ensure the user cannot supply all parts of the path – surround it with your path code
    - Validate the user's input by only accepting known good



- Best Practices of Application Development
  - Use a change control Process
  - Read all related documentation
  - Testing
  - Have a working backup and schedule production downtime
  - Always have a blackout plan
  - Forewarn helpdesk and key user groups
  - Target non-critical servers first
- Prioritizing Findings:
  - Not all findings need to be migrated.
  - You must be in a position to provide - the finding, how the risk was determined, The remediation cost details.
- Testing and Verification
  - When mitigations are implemented, they must be tested, Often part of a promotion between development environments.
- Testing and Verification Roles:
  - Security findings should be addressed the same as any other change request
  - Developer or system owner does not declare the risk mitigated without the concurrence of an independent party.
- Sandbox
  - The sandbox provides a **protective area for program execution**. Limits are placed on the amount of memory and processor resources the program can consume in that sandbox environment.
  - A sandbox cannot confine code and its behavior without some type of enforcement mechanism. The Java security manager makes sure all restricted code stays in the sandbox and cannot ultimately do anything outside of it. Trusted code resides outside the sandbox, and untrusted code is confined within the sandbox. By default, Java applications live outside the sandbox and Java applets are confined within the sandbox.

- **Define and Apply Secure Coding Guidelines and Standards**
  - Source code vulnerabilities
    - Source code vulnerabilities (code weaknesses) can affect functionality, performance, and security
  - **Code Signing Very Important**
    - Code signing is a technique that can be used to address an application's software integrity.
    - Code signing can be used to determine the following:
      - Who is the author of a specific code
      - The purpose or function of the specific code that has been signed
    - Code signing has been shown to provide other benefits. Digital certificates attached to the code can protect users from downloading compromised files or applications that may have malware.
  - Here's an example, let's say an application or program has been signed by a particular developer, but it has been modified after publication. A browser may display a pop-up warning that the code may have been modified and, therefore, cannot be verified as being authentic.
  - Code signing makes all this possible, and it is a tool to allow verification of code as being authentic and that it hasn't been modified
  - Code signing can achieve three distinct purposes. It can be used to:
    - Allow detection of code that may have been altered
    - Identify the author of a specific code or the signer of that code
    - Allow determination of the specific purpose of code, and the reason it was written
  - Code Signature Components
    - To allow code signing to work properly and achieve its goals, code signatures consist of three parts:
      - A seal that can be used to detect unauthorized alterations to the code
      - A digital signature that signs the seal to guarantee its integrity and addresses nonrepudiation of the author
      - A unique identifier used to identify the purpose of the code or to determine which classification the code belongs to
  - **Code Signature Limitations**
    - As with any other security control, code signing is not perfect, and it cannot achieve everything. This is a summary of what code signing may not be able to do:
      - Cannot guarantee that code is free of security vulnerabilities, exploits, or bugs
      - Cannot guarantee that an application or program during execution will not load unsafe or altered code, such as untrusted plug-ins
      - Cannot address Digital Rights Management (DRM) or provide copy protection capabilities





- Secure coding Practices
  - Is the practice of following secure coding standards coupled with the use of testing tools to detect code vulnerabilities.
  - For applications to be designed and implemented with proper security requirements, secure coding practices and a focus on security risks must be integrated into day-to-day operations and the development processes.
  - Three precursors to secure coding are application security training, defining requirements, and threat modeling.
- SEI CERT Top 10 Securing
  - Coding Practices
  - Validate input
  - Heed compiler warning (use the highest warning level available)
  - Architect and design for security policies
  - Keep it simple
  - Default Deny
  - Adhere to the principle of least privilege
  - Sanitize data sent to other systems
  - Practice defense-in-depth
  - Use effective quality assurance techniques
  - Adopt a secure coding standard for your language and platform

Common Software Attacks:

- A buffer overflow attack focuses on sending a computer more input data than it can process
- A Time of Check / Time of Use (TOC/TOU) attack tries to change an operating system condition after it's been checked, but before it's executed
- A redirect and forwarding attack can occur when a web application accepts unvalidated data that redirects a user to an untrusted web address
- Cross-site scripting (XSS) attacks take advantage of 3rd party scripting languages to send unvalidated data to a web application or browser
- A Cross-Site Request Forgery (CSRF) attack focuses on tricking a user to perform unwanted actions on a web application
- An injection attack sends untrusted data to a command interpreter in an attempt to manipulate data
- A drive-by download attack occurs when the software is downloaded without the knowledge of the victim
  A backdoor attack bypasses authorized security controls, communication channels, and other authorized means to access system information